Assignment 2 - Interacting with Web APIs

In [1]:
```
import requests
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Set clean aesthetic for charts
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 5)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [2]:
```
API_KEY = "bd6ea5e490dd29f12dd5a388f72e18ae"
CITY = "Pune"
UNITS = "metric"  # Celsius, m/s

# 5-day / 3-hour forecast endpoint
url = f"https://api.openweathermap.org/data/2.5/forecast?q={CITY}&appid={API_KEY}&units={UNITS}"

response = requests.get(url)

if response.status_code == 200:
    data = response.json()
    print(f"Data retrieved successfully for {CITY}!")
else:
    print(f"Error {response.status_code}: {response.json().get('message')}")
```

```
Data retrieved successfully for Pune!
```

In [3]:
```
# Extract relevant attributes into a list of dictionaries
records = []
for item in data['list']:
    records.append({
        'datetime': item['dt_txt'],
        'temperature': item['main']['temp'],
        'temp_min': item['main']['temp_min'],
        'temp_max': item['main']['temp_max'],
        'humidity': item['main']['humidity'],
        'wind_speed': item['wind']['speed'],
        # precipitation: fallback to 0 if 'rain' key is absent
        'rain_3h': item.get('rain', {}).get('3h', 0.0),
        'weather': item['weather'][0]['main']
    })

# Convert to pandas DataFrame
df = pd.DataFrame(records)

# Data Cleaning & Preprocessing
df['datetime'] = pd.to_datetime(df['datetime'])
df = df.drop_duplicates()
df = df.fillna(0)  # Fill missing values if any

# Add derived time columns for aggregation
df['date'] = df['datetime'].dt.date
df['day_name'] = df['datetime'].dt.day_name()

print(f"Dataset shape: {df.shape}")
df.head()
```

```
Dataset shape: (40, 10)
```

Out[3]:

	datetime	temperature	temp_min	temp_max	humidity	wind_speed	rain_3h	weather	date	day_name
0	2026-10-06 21:00:00	26.30	25.97	26.30	83	0.58	0.0	Clouds	2026-10-06	Tuesday
1	2026-10-07 00:00:00	24.59	23.65	24.59	80	0.56	0.0	Clouds	2026-10-07	Wednesday
2	2026-10-07 03:00:00	27.12	27.12	27.12	58	1.63	0.0	Clear	2026-10-07	Wednesday
3	2026-10-07 06:00:00	32.70	32.70	32.70	34	3.74	0.0	Clear	2026-10-07	Wednesday
4	2026-10-07 09:00:00	34.63	34.63	34.63	28	3.21	0.0	Clear	2026-10-07	Wednesday

Data Modeling & Aggregations

In [4]:
```
# 1. Descriptive Statistics
print("=== Descriptive Statistics ===")
display(df[['temperature', 'humidity', 'wind_speed', 'rain_3h']].describe())

# 2. Daily Summary Aggregations
daily_summary = df.groupby('date').agg(
    avg_temp=('temperature', 'mean'),
    min_temp=('temp_min', 'min'),
    max_temp=('temp_max', 'max'),
    avg_humidity=('humidity', 'mean'),
    total_rain=('rain_3h', 'sum'),
    avg_wind=('wind_speed', 'mean')
).reset_index()

print("\n=== Daily Aggregated Weather Summary ===")
display(daily_summary)
```

```
=== Descriptive Statistics ===
```

	temperature	humidity	wind_speed	rain_3h
count	40.000000	40.000000	40.000000	40.000000
mean	28.846250	54.425000	2.118500	0.003750
std	3.783462	17.674968	1.254362	0.023717
min	23.540000	25.000000	0.330000	0.000000
25%	26.117500	34.750000	1.047500	0.000000
50%	27.535000	59.000000	1.850000	0.000000
75%	32.505000	70.000000	3.107500	0.000000
max	35.820000	83.000000	4.650000	0.150000

```
=== Daily Aggregated Weather Summary ===
```

	date	avg_temp	min_temp	max_temp	avg_humidity	total_rain	avg_wind
0	2026-10-06	26.300000	25.97	26.30	83.000000	0.00	0.580000
1	2026-10-07	30.057500	23.65	35.82	54.000000	0.00	1.958750
2	2026-10-08	29.307500	24.28	35.79	52.500000	0.00	1.947500
3	2026-10-09	27.958750	24.08	33.93	58.125000	0.00	2.625000
4	2026-10-10	28.055000	23.54	34.09	57.000000	0.15	1.912500
5	2026-10-11	29.217143	23.89	34.75	45.857143	0.00	2.372857

Visualizations

In [5]:
```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# 1. Line Chart: Temperature trend over time
axes[0, 0].plot(df['datetime'], df['temperature'], marker='o', color='crimson', linewidth=2)
```

```
axes[0, 0].set_title(f'Temperature Trend Over Time ({CITY})', fontweight='bold')
axes[0, 0].set_ylabel('Temperature (°C)')
axes[0, 0].tick_params(axis='x', rotation=45)

# 2. Bar Chart: Daily Rain Accumulation
axes[0, 1].bar(daily_summary['date'].astype(str), daily_summary['total_rain'], color='royalblue', edgecolor
axes[0, 1].set_title('Total Precipitation by Date', fontweight='bold')
axes[0, 1].set_ylabel('Rain (mm)')
axes[0, 1].tick_params(axis='x', rotation=45)

# 3. Scatter Plot: Temperature vs. Humidity
sns.scatterplot(data=df, x='temperature', y='humidity', hue='weather', s=70, ax=axes[1, 0])
axes[1, 0].set_title('Temperature vs. Humidity', fontweight='bold')
axes[1, 0].set_xlabel('Temperature (°C)')
axes[1, 0].set_ylabel('Humidity (%)')

# 4. Heatmap: Correlation between attributes
corr = df[['temperature', 'humidity', 'wind_speed', 'rain_3h']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f", ax=axes[1, 1])
axes[1, 1].set_title('Weather Attributes Correlation', fontweight='bold')

plt.tight_layout()
plt.show()
```

